

Sprovoznění a použití termovizní kamery WEOM WTC640-N-P-H25-30

Device description

Základní dokumentace v [datovém listu](#) a [výkresu](#)

Po registraci zařízení na stránce podpory výrobce jsem zjistil, že v dokumentaci je nesprávný datasheet, takže jsem musel najít správný na webové stránce podpory

Ve specifikacích bylo uvedeno, že zařízení má USB-C video plugin, ale nebyla to pravda, je tam pouze konektor HDMI – to znamená, že je možné vytvářet pouze statické obrázky a stahovat je přes rozhraní USB nebo zachytávat video přes microHDMI.

Camera

- SN: 20027-048-2412
- AN: WTC640-N-P-H25-30

HDMI plugin

- SN: 20009-074-2412
- AN: P-WTC-H-HDMI

SDK/API

Na webové stránce podpory od dodavatele není k dispozici žádné SDK. Existuje však projekt **Weompy**, na kterém se podílí i společnost Workswell.

Po instalaci `pip install weompy` najdete ve složce `<env>/Lib/site-packages/weompy` dokumentaci v notebooku Jupyter.

Software

Aplikace WEOM GUI – pro ovládání nastavení kamery a pořizování snímků.

CorePlayer – placená aplikace, ale pouze pro kamery s USB-C pluginem, není možné otevřít soubor *.wti, kamera musí být připojena.

Image capturing

Image capturing je pomalý, trvá asi 30 sekund. Výstupní soubor *.wti má nedokumentovaný formát. Jedná se o pole čísel, která představují intenzitu pixelů, ale není jasné, jak přesně se počítá absolutní teplota. V grafickém uživatelském rozhraní je možnost stáhnout soubor palety, ale hodnoty palety jsou namapovány na 256stupňovou stupnici. Je zřejmé, že stupnice je 0–16378, ale není jasné, pro kterou teplotní stupnici. Obrázek neobsahuje radiometrická data. Proto není vhodný pro prahování.

Druhou možností je nakonfigurovat barevné mapování HDMI výstupu kamery a použít nějaký videograber. To byl můj případ, koupil jsem PremiumCord HDMI capture/grabber (EAN 8592220025618).

Camera settings

Možnost konfigurace kamery přes `weompy`.

Připravil jsem malé download/upload [API](#) s konfiguračním souborem v [TOML](#).

Image segmentation

Pro segmentaci jsem zvolil dva algoritmy, oba s optimalizací PSO a bez nutnosti použití ground thue, protože se mi nepodařilo najít žádný datový soubor pro segmentaci pomocí termálních snímků ve stupních šedi bez dat o tepelném záření.

Kapur's entropy

Kapurův algoritmus je globální metoda prahování obrazu, která hledá optimální práh (nebo prahy) tím, že maximalizuje Shannonovu entropii oddělených tříd.

Jinými slovy:

- rozdělení histogramu obrazu na dvě části podle prahu T
- výpočet míry neuspořádanosti (informace) v každé části
- výběr prahu T , který maximalizuje součet entropií obou částí

Entropie je vysoká, když jsou hodnoty ve třídě rovnoměrně rozloženy (více informací).

Pokud je práh „na správném místě“, tak třídy před a za prahem mají maximum informací → to znamená, že práh dobře odděluje dvě různé oblasti obrazu.

Když je práh špatný, jedna třída má málo hodnot a entropie klesá.

Představme si, že histogram obrazu má L úrovní šedé: $0, 1, 2, \dots, L - 1$.

Počty pixelů na jednotlivých úrovních jsou $h_0, h_1, \dots, h_{L-1}$.

Normalizovaný histogram (pravděpodobnosti) je:

$$p_i = \frac{h_i}{\sum_{j=0}^{L-1} h_j}$$

Pokud zvolíme práh T :

- Třída 1: úrovně $0 \dots T$

Pravděpodobnostní distribuce: $p_0, p_1, \dots, p_T$

Součet pravděpodobností: $P_1 = \sum_{i=0}^T p_i$

- Třída 2: úrovně $T + 1 \dots L - 1$

Součet pravděpodobností: $P_2 = \sum_{i=T+1}^{L-1} p_i$

Entropie každé třídy:

$$H_1 = - \sum_{i=0}^T \frac{p_i}{P_1} \ln \frac{p_i}{P_1}$$

$$H_2 = - \sum_{i=T+1}^{L-1} \frac{p_i}{P_2} \ln \frac{p_i}{P_2}$$

Celková entropie:

$$H(T) = H_1 + H_2$$

Optimalizace

- Pro každý možný práh T se vypočte $H(T)$.
- Vybere se takový T^* , pro který je $H(T)$ maximální.

To se dá udělat:

Brute force (pro všechny možné prahy, pokud máš málo úrovní)

Optimalizací (např. PSO, GA, ACO) pro rychlost při větším počtu prahů.

Proč je to dobré na termální snímky

Na termálním snímku mají studené oblasti jiný rozptyl intenzit než horké.

Kapur hledá práh, kde rozdělení pixelů na „studené“ a „horké“ maximalizuje informaci. To obvykle odpovídá místu, kde histogram má přirozený „zlom“.

Block clustering

Tato metoda je inspirována tím, že místo práce s jednotlivými pixely (kterých mohou být miliony v každém snímku) se obraz rozdělí na bloky pevné velikosti (např. 8×8 nebo 16×16 pixelů). Každý blok je pak reprezentován jednou hodnotou – obvykle průměrnou intenzitou (v případě grayscale obrazu) nebo průměrem přes barevné kanály. Tím se počet dat dramaticky sníží.

Rozdělení obrazu na bloky

Nechť máme obraz $I \in \mathbb{R}^{H \times W}$, kde H je výška a W šířka obrazu.

Obraz rozdělíme na bloky o velikosti $B \times B$. Celkový počet bloků je:

$$N = \frac{H}{B} \cdot \frac{W}{B}.$$

Každý blok k reprezentujeme jednou hodnotou (průměrná intenzita):

$$m_k = \frac{1}{B^2} \sum_{i=1}^B \sum_{j=1}^B I_k(i, j),$$

kde $I_k(i, j)$ jsou pixely v bloku k .

Dostaneme tedy vektor průměrů:

$$\mathbf{m} = (m_1, m_2, \dots, m_N).$$

Clustering bloků

Chceme rozdělit bloky do C clusterů (typicky $C = 2$: horké/studené).

Označme centroidy $\mathbf{c} = (c_1, c_2, \dots, c_C)$.

Každý blok m_k se přiřadí ke clusteru:

$$\text{assign}(m_k) = \arg \min_{j \in \{1, \dots, C\}} |m_k - c_j|.$$

Optimalizační problém

Klasické clusteringové kritérium (jako v k-means) minimalizuje tzv. **sum of squared errors (SSE)**:

$$J(\mathbf{c}) = \sum_{k=1}^N \min_{j \in \{1, \dots, C\}} (m_k - c_j)^2.$$

Naším cílem je nalézt centroidy $\mathbf{c}^*$, které minimalizují J :

$$\mathbf{c}^* = \arg \min_{\mathbf{c}} J(\mathbf{c}).$$

Použití PSO pro hledání centroidů

Místo klasického iteračního k-means použijeme **heuristickou optimalizaci** PSO a fitness funkci:

$$\text{fitness}(\mathbf{c}) = -J(\mathbf{c}),$$

protože PSO maximalizuje fitness.

Výsledná maska

Po nalezení optimálních centroidů $\mathbf{c}^*$ se každý blok přiřadí ke clusteru. Typicky vybereme „horký cluster“:

$$\text{hot} = \arg \max_j c_j,$$

a výstupní maska $M \in \{0, 255\}^{H \times W}$ je definována takto:

$$M(i, j) = \begin{cases} 255 & \text{pokud blok } (i, j) \in \text{cluster hot,} \\ 0 & \text{jinak.} \end{cases}$$

Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) je přírodou inspirovaný optimalizační algoritmus, který byl poprvé představen Jamesem Kennedym a Russellem Eberhartem v roce 1995. Inspiraci čerpá z kolektivního chování živočichů, zejména hejn ptáků nebo rojů hmyzu, které hledají potravu v prostoru.

Principem PSO je to, že každá částice (angl. particle) představuje kandidátní řešení v prostoru hledání. Celé hejno částic se pohybuje v tomto prostoru a iterativně upravuje svou pozici podle dvou hlavních faktorů:

Vlastní zkušenost – každá částice si pamatuje nejlepší řešení, které dosud sama objevila, tzv. personal best nebo p_{best} .

Kolektivní zkušenost – částice sleduje také nejlepší řešení nalezené celým rojem, tzv. global best nebo g_{best} .

Pohyb částice je určen její rychlostí, která se v každé iteraci aktualizuje podle rovnice:

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (p_{best,i} - x_i(t)) + c_2 \cdot r_2 \cdot (g_{best,i} - x_i(t))$$

$$x_i(t+1) = x_i(t) + v_i(t+1)$$

Kde:

x_i = pozice částice

v_i = rychlost částice

w = váha setrvačnosti, která udržuje pohyb částice

c_1, c_2 = koeficienty učení (určují vliv vlastní a kolektivní zkušenosti)

r_1, r_2 = náhodná čísla z intervalu $[0, 1]$

Díky těmto rovnicím částice oscilují mezi svým osobním nejlepším řešením a globálním nejlepším řešením, což umožňuje kombinovat **exploraci** (prozkoumávání prostoru) a **exploataci** (zpřesňování řešení).

Vlastnosti

Je jednoduchý na implementaci – stačí pracovat s vektory pozic a rychlostí.

Je stochastický algoritmus – obsahuje náhodné složky, takže může procházet různými oblastmi prostoru.

Je vhodný pro kontinuální optimalizaci, ale existují i varianty pro diskrétní problémy.

Může snadno pracovat s více dimenzemi (každá částice má souřadnice ve vícerozměrném prostoru).

Aplikace

Pro vyzkoušení jsem vytvořil [skript](#), který slouží jako ukázka segmentací.

Existují tedy dva "procesory" `PSOKapurEntropyProcessor` a `PSOBlockClustering`. Jednotlivé způsoby lze různě nastavovat a řadit za sebou, nicméně volání více metod je velmi náročné.